ANÁLISE E PROJETO DE SISTEMAS · TRABALHO DE PESQUISA SOBRE DESIGN PATTERNS

Design Patterns

Padrões criacionais e comportamentais, com exemplos no aplicativo de autoatendimento do food truck Iron Burguer

COMPONENTES DO GRUPO

Gabriel Francisco Pires de Assis
Guilherme Mendes Carlos
Pedro Rebello Borges de Barros
Pedro Vargas dos Santos e Silva

DISCIPLINA

Análise e Projeto de Sistemas
Prof. Eduardo Castro, MSc.
UniCEUB · 2º semestre de 2026

ESCOLHAS DO GRUPO

Categorias e padrões selecionados

CATEGORIA 1

Criacionais

Como os objetos são criados.

Prototype | Singleton

CATEGORIA 2

Comportamentais

Como os objetos interagem e dividem responsabilidades.

Command | State

1

Conceitos

Padrão de projeto e as categorias do catálogo GoF

2

Criacionais

Prototype e Singleton: objetivo, contexto, problema, solução e exemplo

3

Comportamentais

Command e State: objetivo, contexto, problema, solução e exemplo

4

Síntese

Quadro comparativo dos quatro padrões e referências

Os exemplos usam o modelo do **aplicativo Iron Burguer**, desenvolvido na disciplina: as suas classes, os casos de uso (UC) e as regras de negócio (RNG).

CONCEITOS

O que é um padrão de projeto

Um padrão descreve uma **solução para um problema que ocorre com frequência** no desenvolvimento de software: um par **problema/solução** que pode ser reaplicado em contextos diferentes.

NÃO É CÓDIGO PRONTO

Não é classe nem biblioteca para plugar no sistema: é um modelo, uma receita de como organizar classes e objetos.

ORIGEM: GOF

Catálogo de 23 padrões de Gamma, Helm, Johnson e Vlissides, a *Gang of Four*, reunidos a partir de soluções testadas na indústria.

POR QUE USAR

Reuso, manutenção mais fácil, vocabulário comum na equipe e flexibilidade para mudanças.

Elementos da descrição GoF: **nome, problema, solução, estrutura, colaborações e consequências**. Neste trabalho, cada padrão é apresentado por **objetivo, contexto, problema e solução**, seguido de um exemplo.

CONCEITOS

As categorias do catálogo GoF

CRIACIONAIS · ESCOLHIDA

Criação dos objetos: como instanciá-los de forma flexível e controlada.

ESTRUTURAIS

Composição: como classes e objetos se combinam para formar estruturas maiores.

COMPORTAMENTAIS · ESCOLHIDA

Interação: como os objetos se comunicam e dividem as responsabilidades.

Escopo	Criação	Estrutura	Comportamento
Classe	Factory Method	Adapter	Interpreter, Template Method
Objeto	Abstract Factory, Builder, **Prototype**, **Singleton**	Adapter, Bridge, Composite, Decorator, Façade, Flyweight, Proxy	Chain of Responsibility, **Command**, Iterator, Mediator, Memento, Observer, **State**, Strategy, Visitor

Escopo de classe: o padrão usa herança, definida na compilação. **Escopo de objeto:** usa composição, que pode mudar na execução. Os quatro padrões escolhidos são de objeto.

CATEGORIA 1 · CRIACIONAIS

Padrões criacionais

Lidam com a **criação de objetos**: abstraem o processo de instanciação e controlam como os objetos são criados, aumentando a flexibilidade e o reuso. O código que usa um objeto deixa de depender da classe concreta que o cria.

Prototype

Especifica os tipos de objetos a criar usando uma **instância protótipo** e cria novos objetos **copiando esse protótipo**.

No Iron Burguer: cadastrar um produto a partir de outro (UC01).

Singleton

Garante que uma classe tenha **uma única instância** e fornece um **ponto de acesso global** a ela.

No Iron Burguer: a fila de produção única (UC12 e UC13).

Outros padrões da categoria: Abstract Factory · Builder · Factory Method

CRIACIONAL · ESCOPO DE OBJETO

Prototype

OBJETIVO

Criar novos objetos **copiando uma instância já configurada**, o protótipo, em vez de construí-los do zero.

CONTEXTO

Objetos que diferem pouco entre si e têm configuração trabalhosa, ou classes que só são conhecidas na execução.

PROBLEMA

Para criar um objeto parecido com outro, o cliente precisa conhecer a classe concreta e **repetir toda a configuração**, inclusive a dos objetos contidos nele.

SOLUÇÃO

Declarar **clonar()** em uma interface comum. Cada classe concreta sabe copiar a si mesma; o cliente pede o clone e ajusta só o que muda.

Participantes: Prototype, a interface com clonar() · ConcretePrototype, que implementa a cópia · Client, que pede o clone ao protótipo

EXEMPLO · PROPOSTA PARA O UC01 CADASTRAR PRODUTO

Prototype no Iron Burguer

Todo produto tem preço e **ficha técnica** com ingredientes e quantidades (RNG02). Os hambúrgueres de peixe, frango e boi (RNG01) têm fichas parecidas: o novo nasce do clone de um já cadastrado.

O PROTÓTIPO SE COPIA

```
class Hamburguer extends Produto
        implements Prototipo<Hamburguer> {
    private TipoCarne tipoCarne;
    private List<FichaTecnica> ficha;
    public Hamburguer clonar() {
        Hamburguer copia =
            new Hamburguer(nome, preco, tipoCarne);
        for (FichaTecnica f : ficha)
            copia.ficha.add(f.clonar());
        return copia;
    }
}
```

O GERENTE PARTE DO CLONE

```
// UC01: incluir a partir de um existente
Hamburguer frango = boi.clonar();
frango.setNome("Hambúrguer de frango");
frango.setTipoCarne(TipoCarne.FRANGO);
frango.trocarIngrediente(carneBoi,
                         carneFrango);
controladorProduto.incluir(frango);
// grava e exibe MSG01
```

Ganho: o produto novo parte de uma ficha técnica completa, e o Gerente troca só o que muda. **Cuidado:** a cópia tem de ser profunda; com a ficha compartilhada, trocar a carne do frango mudaria a do boi.

CRIACIONAL · ESCOPO DE OBJETO

Singleton

OBJETIVO

Garantir que uma classe tenha **uma única instância** e fornecer um **ponto global de acesso** a ela.

CONTEXTO

Recursos que precisam ser únicos e são usados por várias partes do sistema: uma fila, um contador, uma configuração.

PROBLEMA

Nada impede que cada parte do sistema crie a sua instância com **new**. As cópias divergem, e uma variável global não impede novas instâncias.

SOLUÇÃO

A classe controla a própria instância: **construtor privado**, atributo estático e **getInstancia()**, que cria o objeto na primeira chamada e depois devolve sempre o mesmo.

Participantes: Singleton, que guarda a instância e oferece getInstancia() · Clientes, que obtêm a instância e nunca a criam

EXEMPLO · UC12 REGISTRAR PAGAMENTO E UC13 CONTROLAR FILA DE PRODUÇÃO

Singleton no Iron Burguer

O pedido só entra na fila após a **confirmação do pagamento** (RNG20) e é produzido nessa ordem (RNG23).

A CLASSE CONTROLA A INSTÂNCIA

```
public final class FilaProducao {
    private static FilaProducao instancia;
    private final List<Pedido> fila = new ArrayList<>();

    private FilaProducao() { }

    public static synchronized
            FilaProducao getInstancia() {
        if (instancia == null)
            instancia = new FilaProducao();
        return instancia;
    }
}
```

QUEM USA NÃO CRIA

```
// UC12: pagamento confirmado (RNG20)
FilaProducao.getInstancia()
    .enfileirar(pedido);

// UC13: tela da Produção
List<Pedido> fila =
    FilaProducao.getInstancia()
        .listarFila();
if (fila.isEmpty())
    exibir(MSG22);
```

Ganho: o Cliente e a Produção enxergam a mesma fila, na ordem do pagamento, o que ataca o P8, a fila não respeitada.
Cuidado: estado global dificulta testes, e acessos simultâneos exigem sincronização.

CATEGORIA 2 · COMPORTAMENTAIS

Padrões comportamentais

Focam nos **algoritmos e na atribuição de responsabilidades** entre os objetos, descrevendo como eles interagem e distribuem responsabilidades durante a execução.

Command

Encapsula uma **solicitação como um objeto**, permitindo parametrizar clientes, enfileirar ou registrar solicitações e **desfazer** operações.

No Iron Burguer: a baixa de estoque desfeita no cancelamento (UC17 e UC18).

State

Permite que um objeto **altere o seu comportamento quando o seu estado interno muda**. O objeto parecerá ter mudado de classe.

No Iron Burguer: as situações do pedido (UC11, UC14 e UC15).

Outros padrões da categoria: Chain of Responsibility · Interpreter · Iterator · Mediator · Memento · Observer · Strategy · Template Method · Visitor

COMPORTAMENTAL · ESCOPO DE OBJETO

Command

OBJETIVO

Encapsular uma **solicitação como um objeto**, que pode ser enfileirado, registrado e desfeito.

CONTEXTO

Operações que precisam ser guardadas para depois: executadas em fila, registradas em histórico ou desfeitas.

PROBLEMA

Quem solicita chama o executor diretamente. A operação termina na chamada: **não sobra nada** para enfileirar, registrar ou desfazer.

SOLUÇÃO

Uma interface com **executar()** e **desfazer()**. Cada comando guarda o receptor e os dados de que precisa; um invocador executa e mantém o histórico.

Participantes: Command · ConcreteCommand · Receiver, que faz o trabalho · Invoker, que executa e guarda · Client, que cria o comando

EXEMPLO · UC17 BAIXAR ESTOQUE DE INGREDIENTES E UC18 REPOR ESTOQUE DE INGREDIENTES

Command no Iron Burguer

No pagamento, os ingredientes são baixados pela ficha técnica (UC12 inclui UC17, RNG28). Se um pedido pago for cancelado, eles voltam ao estoque (UC18 estende UC11, RNG22).

A BAIXA VIRA UM OBJETO

```
class BaixarEstoque implements Comando {
    private final Map<Ingrediente, Double> consumo;
    BaixarEstoque(Pedido pedido) {
        consumo = pedido.consumoPelaFicha();
    }
    public void executar() {          // RNG28
        consumo.forEach(Ingrediente::baixarEstoque);
    }
    public void desfazer() {          // RNG22
        consumo.forEach(Ingrediente::reporEstoque);
    }
}
```

O INVOCADOR GUARDA O HISTÓRICO

```
// UC12: pagamento confirmado
Comando c = new BaixarEstoque(pedido);
c.executar();
historico.put(pedido, c);

// UC11: cancela pedido pago (UC18)
historico.remove(pedido).desfazer();
```

Ganho: o comando guarda o que foi baixado, e a reposição devolve as mesmas quantidades. **Cuidado:** só se desfaz o que a regra permite; sem reembolso (RNG21), o pagamento não tem desfazer().

COMPORTAMENTAL · ESCOPO DE OBJETO

State

OBJETIVO

Permitir que um objeto **altere o seu comportamento quando o seu estado interno muda**. Ele parecerá ter mudado de classe.

CONTEXTO

Objetos com ciclo de vida em estados, em que cada operação é permitida ou não conforme o estado atual.

PROBLEMA

Condicionais por estado repetidas em cada operação. A regra fica espalhada, e esquecer um caso gera um comportamento inválido.

SOLUÇÃO

Cada estado vira **uma classe**. O objeto **delega** a operação ao estado atual, e o estado define para qual outro ele passa.

Participantes: Context, o objeto que muda de estado · State, a interface comum · ConcreteState, uma classe para cada estado

EXEMPLO · UC11, UC14 E UC15: CANCELAR, ATUALIZAR A SITUAÇÃO E ENTREGAR O PEDIDO

State no Iron Burguer



SEM O PADRÃO

```
void entregar() {
    if (situacao == PRONTO)
        situacao = ENTREGUE;
    else if (situacao == ENTREGUE)
        erro(MSG24);
    else
        erro(MSG16);
}
// e o mesmo if em cancelar(),
// iniciarProducao(), concluir()...
```

COM O PADRÃO: O PEDIDO DELEGA

```
abstract class Situacao {          // padrão: recusa
    void cancelar(Pedido p) { p.erro(MSG16); }
    void entregar(Pedido p) { p.erro(MSG16); }
}
class Pronto extends Situacao {
    void entregar(Pedido p) { p.mudarPara(new Entregue()); }
}
class Entregue extends Situacao {
    void entregar(Pedido p) { p.erro(MSG24); }   // RNG26
}
```

Ganho: cada regra fica em um só lugar: só Pronto entrega (RNG25), e Entregue recusa a segunda entrega (RNG26). **Custo:** uma classe por situação no lugar da enumeração SituacaoPedido.

SÍNTESE

Os quatro padrões no Iron Burguer

Padrão	Categoria	Objetivo	Aplicação	Base no modelo
Prototype	Criacional	Criar objetos copiando um protótipo	Incluir um produto a partir de outro, com a ficha técnica	UC01 · RNG02
Singleton	Criacional	Uma única instância, com acesso global	Fila de produção única para o Cliente e a Produção	UC12, UC13 · RNG20, RNG23
Command	Comportamental	Solicitação como objeto, que pode ser desfeita	Baixa de estoque desfeita no cancelamento de pedido pago	UC17, UC18 · RNG22, RNG28
State	Comportamental	Comportamento que muda com o estado	Situações do pedido e as operações de cada uma	UC11, UC14, UC15 · RNG17, RNG24 a RNG26

CONCLUSÃO

Cada padrão entrou onde o modelo já tinha o problema. Sem o problema, um padrão só acrescenta classes.

REFERÊNCIAS

GAMMA, E. et al. *Padrões de projeto: soluções reutilizáveis de software orientado a objetos*. Porto Alegre: Bookman, 2007.
CASTRO, E. *Padrões de projeto*. Material da disciplina Análise e Projeto de Sistemas, UniCEUB.